

Analysis of Algorithms

Characteristics of an algorithm

- :-
- • Must take an input.
- • Must give some output (yes/no,value etc.)
- • Definiteness –each instruction is clear and unambiguous.
- • Finiteness –algorithm terminates after a finite number of steps.
- • Effectiveness –every instruction must be basic i.e. simple instruction.

Expectation from an algorithm

- **Correctness:-**
- **Correct:** Algorithms must produce correct result.
 - Produce an incorrect answer: Even if it fails to give correct results all the time still there is a control on how often it gives wrong result.
 - Approximation algorithm: Exact solution is not found, but near optimal solution can be found out. (Applied to optimization problem.)
- **Less resource usage:** Algorithms should use less resources (time and space).

Resource usage

- Here, the time is considered to be the primary measure of efficiency . We are also concerned with how much the respective algorithm involves the computer memory. But mostly time is the resource that is dealt with. And the actual running time depends on a variety of backgrounds: like the speed of the Computer, the language in which the algorithm is implemented, the compiler/interpreter, skill of the programmers etc. So, mainly the resource usage can be divided into:
 1. Memory (space)
 2. Time

Performance Evaluation

This is done as follows

1. How long the algorithm takes:-will be represented as a function of the size of the input. $f(n) \rightarrow$ how long it takes if 'n' is the size of input.
2. How fast the function that characterizes the running time grows with the input size.

Algorithm Analysis

- Understanding the efficiency of an algorithm is important. The speed and/or responsiveness of a wide-variety of applications depend on the efficiency of the algorithm used in the application. Efficient algorithms are much more important than coding tricks and optimization. Examples of applications that rely on efficient algorithms include computer games, productivity software
- We considered the human cost of algorithms (readability, modularity, etc.) before. This impacts how easy it is to generate, modify, test, and support programs, rather than speed or responsiveness for the user. Of course, efficiency is also important. You should always consider the efficiency when selecting a particular algorithm to solve a problem

Reasons to analyze the efficiency of an algorithm:

- Analysis helps choose which solution to use to solve a particular problem.
- Experimentation tells us about one test case, while analysis can tell us about all test cases (performance guarantees).
- Performance can be predicted before programming. If you wait until you've coded a large project and then discover that it runs slowly, much time has been wasted.
- If you can tell which parts of your solution execute quickly and which parts execute slowly, then you know which to work on to improve the overall solution.

Growth rates

- We want to analyze the running time without knowing the details of the input. This is done by measuring the running time as a function of the size of the problem (for example, the value of an integer input, the number of elements in an array, or the number of nodes in a linked list). The size is usually called n , although any variable can be used to denote this size. Let's say we have 3 algorithms to solve some problem. One takes time proportional to $100n$, one takes time proportional to $10n^2$, and one takes time proportional to 2^n . These are the growth rates for the algorithms in terms of the size of the problem n . Let's examine how long these algorithms take for various problem sizes, assuming that times are all in milliseconds

	Algorithm. 1	Algorithm. 2	Algorithm. 3
$n = 5$	500	250	32
$n = 10$	1000	1000	1024
$n = 20$	2000	4000	1,048,576
$n = 50$	5000	25,000	1.1259×10^{15}
$n = 1000$	100,000	10,000,000	$> 10^{300}$

- Algorithm 3 is the fastest for a very small problem, but all of the algorithms are done in under a second. For a slightly larger problem, they are all about the same. As the size of the problem gets bigger, algorithm 3 becomes completely unreasonable and algorithm 1 looks better and better. Usually, it is the performance of algorithms for large problems that we are concerned about, since small problem can be solved quickly using most algorithms. The rate of growth is the primary measurement we will use to analyze efficiency. **Clearly, the rate of growth is the largest for algorithm 3 and lowest for algorithm 1. Algorithm 1 is the best to use if you don't know the size of the problem in advance.** We will examine growth rates using Big-O notation

Big-O notation

- Informally, if the time required by an algorithm is proportional to some function $f(n)$, then the algorithm's running time is said to be $O(f(n))$, where $f(n)$ is the growth-rate function. This is called Big-O notation due to the O (short for order) at the beginning of the notation. Since the running time of algorithm 2 is proportional to n^2 , its running time is $O(n^2)$. $O(n)$ is usually spoken as “big-oh of n ” or simply “oh of n ”.

- **Examples:**

$8n + 10n^3 + 35$ is: $O(n^3)$

$\frac{1}{4} n^2 + 25n \log n$ is: $O(n^2)$

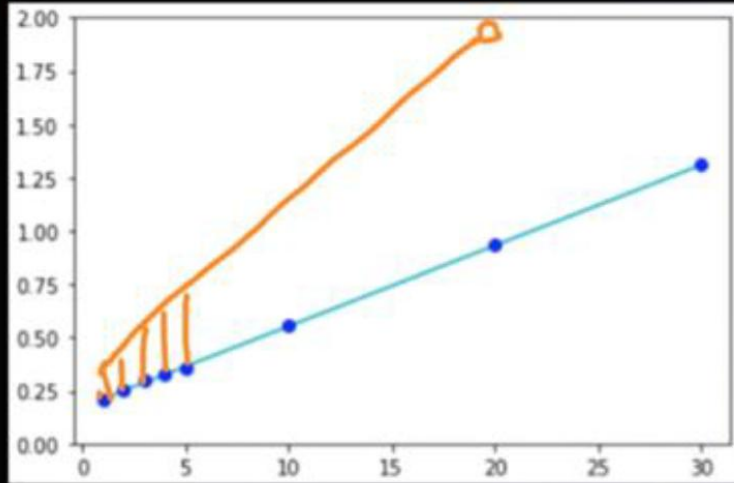
$5n + 2^n / 5$ is: $O(2^n)$

For algorithms, the complexity of the algorithm refers to the rate of growth, so it common to say that the complexity of an algorithm is $O(\text{something})$. The smallest possible complexity for algorithms is $O(1)$, since at least some constant amount of time must be spent no matter what

The hierarchy of complexities is:

- **$O(1)$: Essentially constant.** The complexity can be bounded, no matter how big n is. Example: adding 10 to an integer, accessing one array element.
 - **$O(\log n)$: Logarithmic growth.** This complexity grows very slowly. The base doesn't matter. Example: binary search of a sorted list.
 - **$O(n)$: Linear growth.** Example: traversing a linked list with n nodes, or sequential search of an array.
 - **$O(n \log n)$: Slightly worse than linear.** Usually occurs when you break a problem down into two problems of half the size that must both be solved. Example: Merge sort.
 - **$O(n^2)$: Quadratic growth.** Grows with the square of the problem size. Often two nested loops. Example: Insertion sort. **$O(n^3)$: Cubic growth.** Worse than quadratic. Often three nested loops. Example: Matrix multiplication (straightforward approach).
 - **$O(2^n)$: Exponential growth.** Usually too slow for large problems. Example: The towers of Hanoi.
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

6



linear time

$O(n)$

constant time

$O(1)$

quadratic time

$O(n^2)$

$$T = \underline{an} + b = O(n)$$

$$T = cn^2 + dn + e = O(n^2)$$

1. find the fastest growing term
2. take out the coefficient

Complexity	Example Task	Scalability
$O(1)$	Array access	Excellent
$O(\log n)$	Binary search	Very good
$O(n)$	Loop through list	Good
$O(n \log n)$	Sorting	Efficient
$O(n^2)$	Nested loops	Poor for large data

Big-O helps you:

1. Choose efficient algorithms
2. Predict scalability
3. Optimize performance in large systems

Big-O Complexity Chart

